



Training Guide — Chapter 23

STL Containers in C++

Overview · Specific Containers · Iterators · Algorithms · Adaptors · Allocators

CHAPTER 23: STL CONTAINERS IN C++

23.1 Overview of Containers

23.2 Specific Containers: vector, list, map, queue, stack, deque

23.3 Iterators

23.4 STL Algorithms

23.5 Adaptors

23.6 Allocators

Style: All examples use `#include <iostream>` and idiomatic modern C++ (C++11/14/17). Every program is complete and runnable.



Table of Contents

- [23.1 Overview of Containers](#)
- [23.2 Specific Containers](#)
 - [vector — Dynamic Array](#)
 - [list — Doubly Linked List](#)
 - [map — Key-Value Mapping](#)
 - [queue — FIFO](#)
 - [stack — LIFO](#)
 - [deque — Double-Ended Queue](#)
- [23.3 Iterators](#)
- [23.4 STL Algorithms](#)
- [23.5 Adaptors](#)

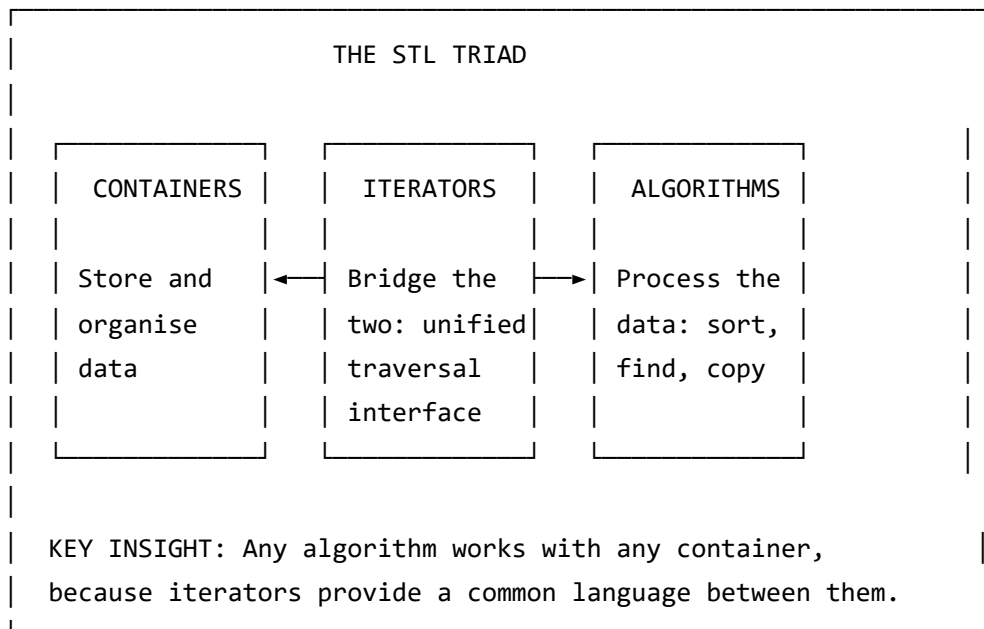
23.1 Overview of Containers

Learning Objectives

- Understand what the STL is and the problem it solves
- Know the three categories of STL containers and when to use each
- Select the right container for a given task based on complexity requirements
- Understand the consistent interface design that makes STL containers interchangeable

What Is the STL?

The **Standard Template Library (STL)** is a collection of ready-made, high-quality, generic components built into C++. It consists of three tightly integrated pillars:



Before the STL, every project reinvented linked lists, hash tables, and sorting from scratch. The STL provides battle-tested, zero-overhead implementations of all standard data structures as **class**

templates — meaning they work for any element type without sacrificing performance.

Three Categories of Containers

STL CONTAINER TAXONOMY

SEQUENCE CONTAINERS

Elements stored in a linear order you control.

You decide where each element goes.

Examples: vector, list, deque, array, forward_list

ASSOCIATIVE CONTAINERS

Elements stored by KEY in a sorted tree structure.

Automatically ordered. Very fast $O(\log n)$ lookup.

Examples: map, set, multimap, multiset

UNORDERED ASSOCIATIVE CONTAINERS

Elements stored by KEY in a hash table.

No ordering. $O(1)$ average lookup – fastest search.

Examples: unordered_map, unordered_set

Complexity Comparison Table

Container	Access by idx	Insert front	Delete middle	Search (unsorted)	Memory
vector	$O(1)$ ★	$O(n)$	$O(n)$	$O(n)$	Low
deque	$O(1)$	$O(1)$ ★	$O(n)$	$O(n)$	Medium
list	$O(n)$	$O(1)$ ★	$O(1)$ ★	$O(n)$	High
array	$O(1)$ ★	N/A	N/A	$O(n)$	Lowest
map	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$ ★	Medium
set	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$ ★	Medium
unordered_map	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg ★	Higher
unordered_set	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg ★	Higher

★ = best in class for that operation

Container Selection Guide

WHICH CONTAINER SHOULD I USE?

- └─ Do you need random access by index (`arr[i]`)?
 - | YES → `vector` (or `array` if fixed size)
 - | NO ↓
- └─ Do you need fast insert/delete at BOTH ends?
 - | YES → `deque`
 - | NO ↓
- └─ Do you need fast insert/delete in the MIDDLE?
 - | YES → `list`
 - | NO ↓
- └─ Do you need to store key-value pairs with fast lookup?
 - | YES – Do you need sorted order?
 - | YES → `map` (or `multimap` for duplicate keys)
 - | NO → `unordered_map` (faster average)
- └─ Do you need a set of unique values?
 - | YES – Sorted? → `set`; Unsorted? → `unordered_set`
- └─ FIFO queue? → `queue` LIFO stack? → `stack`
 - | Priority ordering? → `priority_queue`

The Common Container Interface

Every STL container shares a consistent set of methods. This is one of the STL's greatest strengths — once you learn one container, you mostly know them all.

METHODS COMMON TO ALL STL CONTAINERS:

<code>c.size()</code>	→ number of elements currently stored
<code>c.empty()</code>	→ true if <code>c.size() == 0</code>
<code>c.clear()</code>	→ remove all elements
<code>c.begin()</code>	→ iterator to the first element
<code>c.end()</code>	→ iterator one-past-the-last (sentinel)
<code>c.swap(other)</code>	→ swap contents with another container of same type

SEQUENCE CONTAINERS ALSO HAVE:

<code>c.push_back(val)</code>	→ add to end
<code>c.pop_back()</code>	→ remove from end
<code>c.front()</code>	→ reference to first element
<code>c.back()</code>	→ reference to last element
<code>c.insert(it, val)</code>	→ insert before iterator position
<code>c.erase(it)</code>	→ erase at iterator position

23.2 Specific Containers

vector — Dynamic Array

Theory

`std::vector<T>` is the workhorse of the STL — the go-to container for most tasks. It stores elements **contiguously in memory** (like a C array) but can grow dynamically. When it runs out of capacity, it allocates a new, larger block (typically 2× the old size) and moves all elements there.

VECTOR MEMORY MODEL:

```
vector<int> v = {10, 20, 30};
```

Logical view:

10	20	30
----	----	----

 size=3, capacity=3

After `v.push_back(40)`: (capacity exceeded → reallocate!)

10	20	30	40		
----	----	----	----	--	--

 size=4, capacity=6 (2× old)

KEY PROPERTIES:

- ✓ $O(1)$ random access: `v[i]`, `v.at(i)`
- ✓ $O(1)$ amortised `push_back` (occasionally $O(n)$ for reallocation)
- ✗ $O(n)$ insert/delete in the middle (elements must shift)
- ✓ Cache-friendly: contiguous memory means CPU prefetcher works well

```

#include <iostream>
#include <vector>
#include <algorithm> // sort, find, for_each
#include <string>
#include <numeric> // accumulate

// — Utility: print a vector —————
template <typename T>
void printVec(const std::vector<T>& v, const std::string& label) {
    std::cout << label << " [" << v.size() << "/" << v.capacity() << "]: ";
    for (const auto& elem : v) std::cout << elem << " ";
    std::cout << "\n";
}

int main() {
    std::cout << "┌───────────────────────────────────┐\n";
    std::cout << "│      std::vector – Complete Demo      │\n";
    std::cout << "└───────────────────────────────────┘\n\n";

    // — Construction —————
    std::cout << "— Construction —\n";
    std::vector<int> v1; // empty
    std::vector<int> v2(5, 42); // 5 elements, all 42
    std::vector<int> v3 = {3, 1, 4, 1, 5, 9, 2, 6}; // initialiser list
    std::vector<int> v4(v3.begin(), v3.begin() + 4); // from range

    printVec(v1, "v1 (empty) ");
    printVec(v2, "v2 (5×42) ");
    printVec(v3, "v3 (literal)");
    printVec(v4, "v4 (range) ");

    // — Push / Pop and capacity growth —————
    std::cout << "\n— Push/Pop and capacity —\n";
    std::vector<int> v;
    std::cout << "Initial capacity: " << v.capacity() << "\n";
    for (int i = 1; i <= 8; i++) {
        v.push_back(i * 10);
        std::cout << "  push_back(" << i*10 << ")  size="
                  << v.size() << " cap=" << v.capacity() << "\n";
    }
    printVec(v, "v after pushes");

    v.pop_back();

```

```

v.pop_back();
printVec(v, "v after 2 pops");

// — Random access —————
std::cout << "\n— Random access —\n";
std::cout << "v[0]   = " << v[0]       << "   (no bounds check)\n";
std::cout << "v.at(2)= " << v.at(2)    << "   (bounds-checked)\n";
std::cout << "front  = " << v.front() << "\n";
std::cout << "back   = " << v.back()  << "\n";

// bounds-checked at() throws on bad index
try { v.at(999); }
catch (const std::out_of_range& e) {
    std::cout << "Caught out_of_range: " << e.what() << "\n";
}

// — Insert and Erase —————
std::cout << "\n— Insert and Erase —\n";
std::vector<int> vi = {10, 20, 40, 50};
printVec(vi, "before insert");

auto it = vi.begin() + 2;           // iterator to index 2 (value 40)
vi.insert(it, 30);                  // insert 30 before 40
printVec(vi, "after insert(30)");

vi.insert(vi.end(), {60, 70, 80}); // insert multiple at end
printVec(vi, "after insert range");

vi.erase(vi.begin());               // erase first element
printVec(vi, "after erase first");

vi.erase(vi.begin() + 1, vi.begin() + 3); // erase range [1,3)
printVec(vi, "after erase range");

// — Reserve and Shrink —————
std::cout << "\n— Reserve and memory management —\n";
std::vector<int> rv;
rv.reserve(100); // pre-allocate for 100 elements – avoids reallocations
std::cout << "After reserve(100): size=" << rv.size()
          << " cap=" << rv.capacity() << "\n";
for (int i = 0; i < 10; i++) rv.push_back(i);
std::cout << "After 10 pushes:   size=" << rv.size()
          << " cap=" << rv.capacity() << "\n";

```

```

rv.shrink_to_fit(); // release unused capacity
std::cout << "After shrink_to_fit:size=" << rv.size()
           << " cap=" << rv.capacity() << "\n";

// — Sort and Search —————
std::cout << "\n— Sort and Search —\n";
std::vector<int> sv = {5, 2, 8, 1, 9, 3, 7, 4, 6};
printVec(sv, "unsorted");
std::sort(sv.begin(), sv.end());
printVec(sv, "sorted  ");

auto found = std::find(sv.begin(), sv.end(), 7);
if (found != sv.end())
    std::cout << "Found 7 at index "
              << std::distance(sv.begin(), found) << "\n";

bool exists = std::binary_search(sv.begin(), sv.end(), 9);
std::cout << "Binary search for 9: " << (exists ? "found" : "not found") << "\n";

// — Aggregate operations —————
std::cout << "\n— Aggregate operations —\n";
int sum = std::accumulate(sv.begin(), sv.end(), 0);
int mn  = *std::min_element(sv.begin(), sv.end());
int mx  = *std::max_element(sv.begin(), sv.end());
std::cout << "Sum=" << sum << " Min=" << mn << " Max=" << mx << "\n";

// — 2D vector (vector of vectors) —————
std::cout << "\n— 2D vector (matrix) —\n";
std::vector<std::vector<int>> matrix(3, std::vector<int>(3, 0));
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
        matrix[i][j] = i * 3 + j + 1;

for (const auto& row : matrix) {
    for (int val : row) std::cout << val << " ";
    std::cout << "\n";
}

return 0;
}

```

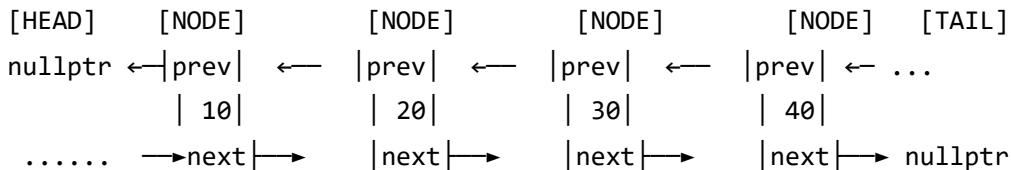
list — Doubly Linked List

Theory

`std::list<T>` is a doubly-linked list. Each element is an independent **node** allocated separately on the heap, containing the value plus pointers to the previous and next nodes.

LIST MEMORY MODEL:

```
list<int> = {10, 20, 30, 40}
```



KEY PROPERTIES:

- ✓ $O(1)$ insert/delete ANYWHERE if you have an iterator (no shifting!)
- ✓ $O(1)$ push_front and push_back
- ✗ $O(n)$ random access – no `[]` operator, must traverse from head
- ✗ Higher memory: each node has two extra pointers (prev + next)
- ✗ Poor cache performance: nodes scattered in heap memory

BEST FOR: Frequent insertion/deletion in the middle;
never needing random access by index.

```

#include <iostream>
#include <list>
#include <string>
#include <algorithm>

void printList(const std::list<int>& lst, const std::string& label) {
    std::cout << label << " [" << lst.size() << "]: ";
    for (int v : lst) std::cout << v << " ";
    std::cout << "\n";
}

int main() {
    std::cout << "||\n";
    std::cout << "||     std::list – Complete Demo     ||\n";
    std::cout << "||\n\n";

    // — Construction —————
    std::list<int> lst = {30, 10, 50, 20, 40};
    printList(lst, "initial");

    // — Front and Back insertion / removal —————
    std::cout << "\n—— Push/Pop front and back ——\n";
    lst.push_front(5);
    lst.push_back(60);
    printList(lst, "after push front(5), back(60)");

    lst.pop_front();
    lst.pop_back();
    printList(lst, "after pop front and back");

    // — Insert in the middle using iterator —————
    std::cout << "\n—— Insert in middle (0(1) once iterator known) ——\n";
    auto it = lst.begin();
    std::advance(it, 2);           // move iterator 2 steps forward
    std::cout << "Iterator points to: " << *it << "\n";
    lst.insert(it, 99);           // insert 99 BEFORE that position
    printList(lst, "after insert(99) at pos 2");

    // — Erase in the middle —————
    it = lst.begin();
    std::advance(it, 1);
    std::cout << "Erasing element: " << *it << "\n";
    lst.erase(it);
}

```

```

printList(lst, "after erase");

// — list-specific algorithms —————
std::cout << "\n— list-specific methods —\n";
std::list<int> lst2 = {100, 200, 300};
printList(lst2, "lst2");

lst.splice(lst.end(), lst2);    // move ALL of lst2 into lst at end
printList(lst, "after splice lst2 into lst");
printList(lst2, "lst2 after splice (now empty)");

// Sort (list has its own sort – std::sort doesn't work on list)
lst.sort();
printList(lst, "after sort");

// Remove specific value (all occurrences)
lst.push_back(30); lst.push_back(30);
printList(lst, "before remove(30)");
lst.remove(30);
printList(lst, "after remove(30)");

// Remove duplicates (list must be sorted first)
lst.push_back(10); lst.push_back(10); lst.push_back(20);
lst.sort();
printList(lst, "before unique");
lst.unique();
printList(lst, "after unique (removes consecutive dups)");

// Reverse
lst.reverse();
printList(lst, "after reverse");

// — Merge two sorted lists —————
std::cout << "\n— Merge two sorted lists —\n";
std::list<int> la = {1, 3, 5, 7};
std::list<int> lb = {2, 4, 6, 8};
printList(la, "la");
printList(lb, "lb");
la.merge(lb);    // merges lb into la; lb becomes empty
printList(la, "la after merge(lb)");
printList(lb, "lb (now empty)");

// — Real-world: task scheduler using list —————

```

```

std::cout << "\n— Real-world: Task Scheduler —\n";
std::list<std::string> tasks;
tasks.push_back("Load config");
tasks.push_back("Connect to DB");
tasks.push_back("Authenticate user");
tasks.push_back("Fetch data");
tasks.push_back("Render UI");

// Insert urgent task at the front
tasks.push_front("URGENT: Check security patch");

std::cout << "Task queue:\n";
int n = 1;
for (const auto& t : tasks)
    std::cout << "  " << n++ << ". " << t << "\n";

// Remove a task that's no longer needed
tasks.remove("Connect to DB");
std::cout << "\nAfter removing 'Connect to DB':\n";
n = 1;
for (const auto& t : tasks)
    std::cout << "  " << n++ << ". " << t << "\n";

return 0;
}

```

map — Key-Value Mapping

Theory

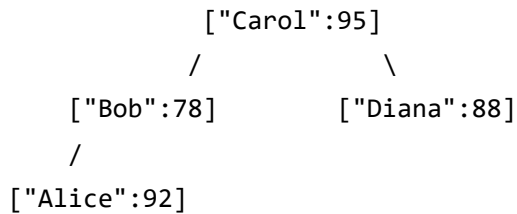
`std::map<K, V>` stores key-value pairs in a **self-balancing binary search tree** (usually a Red-Black tree). Keys are always maintained in sorted order, and every key is unique.

MAP INTERNAL STRUCTURE (Red-Black Tree):

map<string, int> scores:

```
{"Alice":92, "Bob":78, "Carol":95, "Diana":88}
```

Stored in sorted key order (alphabetical for strings):



KEY PROPERTIES:

- ✓ $O(\log n)$ for find, insert, delete – always guaranteed
- ✓ Keys always sorted – iterate to get sorted sequence
- ✓ Unique keys – inserting duplicate key updates/fails, not duplicates
- X Higher overhead than unordered_map (tree nodes, pointers)
- X Slower than hash table for pure lookup ($O(\log n)$ vs $O(1)$ avg)

USE WHEN: You need sorted iteration over keys, or range queries.

USE unordered_map WHEN: You only need fast lookup, no ordering needed.

```

#include <iostream>
#include <map>
#include <string>
#include <algorithm>

void printMap(const std::map<std::string, int>& m, const std::string& label) {
    std::cout << label << " [" << m.size() << " entries]:\n";
    for (const auto& [key, val] : m) // structured bindings (C++17)
        std::cout << "   \"" << key << "\" → " << val << "\n";
}

int main() {
    std::cout << "┌───────────────────────────────────┐\n";
    std::cout << "│           std::map – Complete Demo           │\n";
    std::cout << "└───────────────────────────────────┘\n\n";

    // — Construction and insertion —————
    std::cout << "—— Construction and insertion ——\n";
    std::map<std::string, int> scores;

    // Method 1: operator[] (creates with default if key absent)
    scores["Alice"] = 92;
    scores["Bob"]   = 78;
    scores["Carol"] = 95;

    // Method 2: insert() with pair
    scores.insert({"Diana", 88});
    scores.insert(std::make_pair("Eve", 71));

    // Method 3: emplace() – constructs in-place (most efficient)
    scores.emplace("Frank", 84);

    printMap(scores, "scores");

    // — Accessing elements —————
    std::cout << "\n—— Access ——\n";
    std::cout << "Alice's score: " << scores["Alice"] << "\n";

    // WARNING: operator[] INSERTS a default entry if key doesn't exist!
    // Use find() to check existence without inserting:
    auto it = scores.find("Zara");
    if (it == scores.end())
        std::cout << "Zara not found (no insertion occurred)\n";
}

```

```

// at() is bounds-checked and throws if key absent:
try {
    std::cout << scores.at("Bob") << "\n";    // OK
    std::cout << scores.at("Zara") << "\n";    // throws!
}
catch (const std::out_of_range& e) {
    std::cout << "at() threw: " << e.what() << "\n";
}

// count() returns 0 or 1 for map (keys are unique)
std::cout << "count(\"Carol\") : " << scores.count("Carol") << "\n";
std::cout << "count(\"Zara\") : " << scores.count("Zara") << "\n";

// — Update an existing value —————
std::cout << "\n— Update —\n";
scores["Alice"] = 99;    // update via []
scores.at("Bob") = 85;   // update via at()
printMap(scores, "after updates");

// — Erase —————
std::cout << "\n— Erase —\n";
scores.erase("Eve");
std::cout << "After erase('Eve'):\n";
for (const auto& [k, v] : scores)
    std::cout << " " << k << "=" << v << "\n";

// Erase by iterator range (erase Alice and Bob – first two entries)
auto first = scores.begin();
auto third = scores.begin();
std::advance(third, 2);
scores.erase(first, third);
std::cout << "\nAfter range erase (first 2):\n";
for (const auto& [k, v] : scores)
    std::cout << " " << k << "=" << v << "\n";

// — lower_bound / upper_bound (range queries) —————
std::cout << "\n— Range queries —\n";
std::map<int, std::string> calendar;
calendar[1] = "New Year";
calendar[14] = "Valentine's Day";
calendar[25] = "Christmas";
calendar[31] = "New Year's Eve";

```

```

calendar[17] = "St. Patrick's Day";

// Find all holidays in the range [14, 25]
auto lo = calendar.lower_bound(14); // first key >= 14
auto hi = calendar.upper_bound(25); // first key > 25

std::cout << "Holidays from day 14 to 25:\n";
for (auto i = lo; i != hi; ++i)
    std::cout << "   Day " << i->first << ": " << i->second << "\n";

// — multimap: allows duplicate keys —————
std::cout << "\n— std::multimap (duplicate keys) —\n";
std::multimap<std::string, std::string> contacts;
contacts.insert({"Alice", "alice@work.com"});
contacts.insert({"Alice", "alice@home.com"}); // duplicate key!
contacts.insert({"Bob", "bob@work.com"});

// Find all emails for Alice
auto range = contacts.equal_range("Alice");
std::cout << "Alice's emails:\n";
for (auto i = range.first; i != range.second; ++i)
    std::cout << "   " << i->second << "\n";

// — Real-world: word frequency counter —————
std::cout << "\n— Real-world: Word Frequency —\n";
std::string text = "the cat sat on the mat the cat sat";
std::map<std::string, int> freq;
std::string word;

// Parse words and count
for (char c : text) {
    if (c == ' ') {
        if (!word.empty()) { freq[word]++; word.clear(); }
    } else {
        word += c;
    }
}
if (!word.empty()) freq[word]++;

std::cout << "Word frequencies (sorted alphabetically):\n";
for (const auto& [w, cnt] : freq)
    std::cout << "   \"" << w << "\": " << cnt << " times\n";

```

```
    return 0;  
}
```

queue — FIFO

Theory

`std::queue<T>` is a **First-In, First-Out** container adaptor. Elements enter at the **back** and leave from the **front**, exactly like a real-world queue (line of people).

QUEUE MODEL (FIFO):

enqueue → [D][C][B][A] → dequeue
 BACK FRONT

push("Alice"): [Alice]
push("Bob"): [Bob][Alice]
push("Carol"): [Carol][Bob][Alice]
pop(): returns "Alice" → [Carol][Bob]
front(): returns "Bob" (next to be served)

OPERATIONS: push() O(1)
 pop() O(1)
 front() O(1)
 back() O(1)
 size() O(1)

```

#include <iostream>
#include <queue>
#include <string>

int main() {
    std::cout << "┌───────────────────────────────────┐\n";
    std::cout << "│      std::queue – Complete Demo      │\n";
    std::cout << "└───────────────────────────────────┘\n\n";

    // — Basic operations —————
    std::cout << "—— Basic operations ——\n";
    std::queue<int> q;
    std::cout << "empty? " << (q.empty() ? "yes" : "no") << "\n";

    for (int i = 10; i <= 50; i += 10) {
        q.push(i);
        std::cout << "push(" << i << ")  size=" << q.size()
                  << "  front=" << q.front()
                  << "  back=" << q.back() << "\n";
    }

    std::cout << "\nDraining the queue (FIFO order):\n";
    while (!q.empty()) {
        std::cout << "  pop() → " << q.front() << "\n";
        q.pop();
    }

    // — Real-world: Print Spooler —————
    std::cout << "\n—— Real-world: Print Spooler ——\n";
    struct PrintJob {
        std::string filename;
        int         pages;
        std::string priority;
    };

    std::queue<PrintJob> spooler;
    spooler.push({"report.pdf",    12, "normal"});
    spooler.push({"invoice.pdf",   2,  "urgent"});
    spooler.push({"presentation.ppt", 8, "normal"});
    spooler.push({"letter.docx",   1, "normal"});

    std::cout << "Processing print jobs (FIFO):\n";
    int jobNum = 1;

```

```

while (!spooler.empty()) {
    const PrintJob& job = spooler.front();
    std::cout << "  Job " << jobNum++ << ": ["
                << job.priority << "]" "
                << job.filename << " (" << job.pages << " pages)\n";
    spooler.pop();
}
std::cout << "All jobs printed.\n";

```

// — BFS traversal using queue (graph/tree) —————

```
std::cout << "\n— BFS Level-order traversal using queue —\n";
```

// Adjacency list for a small undirected graph:

// 0-1-3

// | |

// 2-4-5

```

int adj[6][6] = {
    {0,1,1,0,0,0}, // 0 connects to 1, 2
    {1,0,0,1,0,0}, // 1 connects to 0, 3
    {1,0,0,0,1,0}, // 2 connects to 0, 4
    {0,1,0,0,0,1}, // 3 connects to 1, 5
    {0,0,1,0,0,1}, // 4 connects to 2, 5
    {0,0,0,1,1,0}  // 5 connects to 3, 4
};

```

```
bool visited[6] = {false};
```

```
std::queue<int> bfsQ;
```

```
bfsQ.push(0);
```

```
visited[0] = true;
```

```
std::cout << "BFS from node 0: ";
```

```

while (!bfsQ.empty()) {
    int node = bfsQ.front(); bfsQ.pop();
    std::cout << node << " ";
    for (int nb = 0; nb < 6; nb++) {
        if (adj[node][nb] && !visited[nb]) {
            visited[nb] = true;
            bfsQ.push(nb);
        }
    }
}
std::cout << "\n";

```

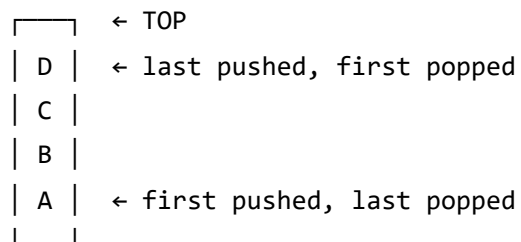
```
    return 0;  
}
```

stack — LIFO

Theory

`std::stack<T>` is a **Last-In, First-Out** container adaptor. Elements are added and removed from the **same end** (the top), like a stack of plates.

STACK MODEL (LIFO):



push(A)→push(B)→push(C)→push(D)→pop()=D→pop()=C

OPERATIONS: push() 0(1)
 pop() 0(1)
 top() 0(1)
 size() 0(1)

```

#include <iostream>
#include <stack>
#include <string>
#include <sstream>

int main() {
    std::cout << "┌───────────────────────────────────┐\n";
    std::cout << "│      std::stack – Complete Demo      │\n";
    std::cout << "└───────────────────────────────────┘\n\n";

    // — Basic operations —————
    std::cout << "—— Basic operations ——\n";
    std::stack<int> s;
    for (int i = 10; i <= 50; i += 10) {
        s.push(i);
        std::cout << "push(" << i << ")  size=" << s.size()
                    << "  top=" << s.top() << "\n";
    }
    std::cout << "\nDraining stack (LIFO order):\n";
    while (!s.empty()) {
        std::cout << "  pop() → " << s.top() << "\n";
        s.pop();
    }

    // — Real-world 1: Balanced brackets checker —————
    std::cout << "\n—— Real-world: Balanced Brackets ——\n";
    auto isBalanced = [](const std::string& expr) -> bool {
        std::stack<char> stk;
        for (char c : expr) {
            if (c == '(' || c == '[' || c == '{') {
                stk.push(c);
            } else if (c == ')' || c == ']' || c == '}') {
                if (stk.empty()) return false;
                char top = stk.top(); stk.pop();
                if ((c == ')' && top != '(') ||
                    (c == ']' && top != '[') ||
                    (c == '}' && top != '{')) return false;
            }
        }
        return stk.empty();
    };

    std::string exprs[] = {"{[(())]}" , "{[()]}" , "((()))" , "((()))" , "()" };

```

```

for (const auto& e : exprs)
    std::cout << "  \"" << e << "\" → "
        << (isBalanced(e) ? "balanced ✓" : "NOT balanced X") << "\n";

// — Real-world 2: Undo history —————
std::cout << "\n— Real-world: Undo History —\n";
struct Action { std::string type; std::string data; };
std::stack<Action> undoStack;

auto doAction = [&](const std::string& type, const std::string& data) {
    undoStack.push({type, data});
    std::cout << " DO:  [" << type << "] \"" << data << "\"\n";
};

auto undoAction = [&]() {
    if (undoStack.empty()) {
        std::cout << " UNDO: nothing to undo.\n";
        return;
    }
    Action a = undoStack.top(); undoStack.pop();
    std::cout << " UNDO: reversed [" << a.type
        << "] \"" << a.data << "\"\n";
};

doAction("type",    "Hello");
doAction("type",    " World");
doAction("delete",  "last word");
doAction("format",  "bold");
undoAction();
undoAction();
undoAction();
undoAction(); // nothing left

// — Real-world 3: Postfix expression evaluator —————
std::cout << "\n— Real-world: Postfix Expression Evaluator —\n";
auto evalPostfix = [](const std::string& expr) -> double {
    std::stack<double> stk;
    std::istringstream iss(expr);
    std::string token;
    while (iss >> token) {
        if (token == "+" || token == "-" ||
            token == "*" || token == "/") {
            double b = stk.top(); stk.pop();

```

```

        double a = stk.top(); stk.pop();
        if      (token == "+") stk.push(a + b);
        else if (token == "-") stk.push(a - b);
        else if (token == "*") stk.push(a * b);
        else if (token == "/") stk.push(a / b);
    } else {
        stk.push(std::stod(token));
    }
}
return stk.top();
};

// "3 4 + 5 *" = (3+4)*5 = 35
std::cout << "\"3 4 + 5 *\"  = " << evalPostfix("3 4 + 5 *") << "\n";
// "2 3 4 * +" = 2 + (3*4) = 14
std::cout << "\"2 3 4 * +\"  = " << evalPostfix("2 3 4 * +") << "\n";
// "5 1 2 + 4 * + 3 -" = 5 + (1+2)*4 - 3 = 14
std::cout << "\"5 1 2 + 4 * + 3 -\" = "
          << evalPostfix("5 1 2 + 4 * + 3 -") << "\n";

return 0;
}

```

deque — Double-Ended Queue

Theory

`std::deque<T>` (pronounced "deck") is a **double-ended queue** — it provides $O(1)$ push and pop at **both** the front and the back. Internally it uses a sequence of fixed-size chunks (not a single contiguous block), giving it both random access and efficient front-insertion.

DEQUE MEMORY MODEL:

deque is stored as a sequence of fixed-size "pages" (chunks):

```
[chunk 0]  [chunk 1]  [chunk 2]
[6][7][8]  [9][0][1]  [2][3][4][5]
```

A central "map" array holds pointers to chunks.

front and back can grow into new chunks.

KEY PROPERTIES:

- ✓ $O(1)$ push_front, push_back, pop_front, pop_back
- ✓ $O(1)$ random access via [] (slightly slower than vector)
- ✗ Higher constant overhead than vector (chunk indirection)
- ✗ Not cache-friendly as vector (chunks not contiguous)

USE WHEN: You need fast insertion at BOTH ends.

std::queue and std::stack use deque as their default underlying container.

```

#include <iostream>
#include <deque>
#include <string>
#include <algorithm>

void printDeque(const std::deque<int>& dq, const std::string& label) {
    std::cout << label << " [" << dq.size() << "]: ";
    for (int v : dq) std::cout << v << " ";
    std::cout << "\n";
}

int main() {
    std::cout << "||\n";
    std::cout << "||    std::deque – Complete Demo    ||\n";
    std::cout << "||\n\n";

    // — Construction and both-end insertion —————
    std::cout << "—— Both-end push ——\n";
    std::deque<int> dq = {30, 40, 50};
    printDeque(dq, "initial");

    dq.push_front(20);
    dq.push_front(10);
    dq.push_back(60);
    dq.push_back(70);
    printDeque(dq, "after push_front(20,10) and push_back(60,70)");

    // — Random access —————
    std::cout << "\n—— Random access (like vector) ——\n";
    std::cout << "dq[0]    = " << dq[0]    << "\n";
    std::cout << "dq.at(3) = " << dq.at(3)    << "\n";
    std::cout << "front() = " << dq.front() << "\n";
    std::cout << "back()  = " << dq.back()  << "\n";

    // — Pop from both ends —————
    std::cout << "\n—— Pop from both ends ——\n";
    dq.pop_front();
    dq.pop_back();
    printDeque(dq, "after pop_front() and pop_back()");

    // — Insert and erase in middle (same as vector, uses iterators) ———
    auto it = dq.begin() + 2;
    dq.insert(it, 999);

```

```

printDeque(dq, "after insert(999) at pos 2");

// — Sort —————
std::sort(dq.begin(), dq.end());
printDeque(dq, "after sort");

// — Real-world: Sliding Window Maximum —————
// A classic algorithm that requires deque's both-end efficiency.
// Problem: given array and window size k, find max in each window.
std::cout << "\n— Real-world: Sliding Window Maximum (k=3) —\n";

std::vector<int> nums = {1, 3, -1, -3, 5, 3, 6, 7};
int k = 3;
std::deque<int> window; // stores INDICES, not values
std::vector<int> results;

std::cout << "Input: ";
for (int n : nums) std::cout << n << " ";
std::cout << "\n";

for (int i = 0; i < (int)nums.size(); i++) {
    // Remove indices that are out of the window
    while (!window.empty() && window.front() < i - k + 1)
        window.pop_front();

    // Remove indices of elements smaller than current (from back)
    while (!window.empty() && nums[window.back()] < nums[i])
        window.pop_back();

    window.push_back(i);

    // Start recording once we have a full window
    if (i >= k - 1)
        results.push_back(nums[window.front()]);
}

std::cout << "Max(k=3): ";
for (int r : results) std::cout << r << " ";
std::cout << "\n";

// — Real-world: Undo/Redo buffer —————
std::cout << "\n— Real-world: Undo/Redo with deque —\n";
std::deque<std::string> history;

```

```

int historyPos = -1;

auto execute = [&](const std::string& cmd) {
    // Remove any redo future if we execute a new command
    while ((int)history.size() > historyPos + 1)
        history.pop_back();
    history.push_back(cmd);
    historyPos++;
    std::cout << "Execute: \"" << cmd << "\"\n";
};

auto undo = [&]() {
    if (historyPos < 0) { std::cout << "Nothing to undo.\n"; return; }
    std::cout << "Undo:   \"" << history[historyPos] << "\"\n";
    historyPos--;
};

auto redo = [&]() {
    if (historyPos >= (int)history.size()-1)
        { std::cout << "Nothing to redo.\n"; return; }
    historyPos++;
    std::cout << "Redo:   \"" << history[historyPos] << "\"\n";
};

execute("Type 'Hello'");
execute("Type ' World'");
execute("Bold selection");
undo();
undo();
redo();
execute("Italic selection"); // new branch – removes 'Bold' redo
undo();

return 0;
}

```

23.3 Iterators

Theory

An **iterator** is an object that acts as a pointer into a container. It provides a uniform way to traverse any STL container using the same operations (`*` , `++` , `--` , etc.), regardless of the container's internal structure.

THE ITERATOR ABSTRACTION:

Without iterators – algorithm must know container internals:

```
for (int i = 0; i < vec.size(); i++) process(vec[i]);    // vector-specific
for (Node* p = head; p; p = p->next) process(p->data);  // list-specific
```

With iterators – ONE algorithm works for ANY container:

```
for (auto it = c.begin(); it != c.end(); ++it) process(*it);
// Same code for vector, list, deque, set, map, or your own container!
```

KEY ITERATOR OPERATIONS:

```
*it      → dereference: get the element at this position
++it     → advance to the next element (prefix form preferred)
it++     → advance, return old position (postfix form – creates copy)
--it     → move backward (bidirectional iterators only)
it + n   → advance by n steps (random access iterators only)
it1==it2 → are both iterators at the same position?
it1!=it2 → are they at different positions?
```

Iterator Hierarchy

ITERATOR TYPES (weakest to strongest – each adds capabilities):

INPUT ITERATOR

- Read-only, single-pass (like reading from a file)
- Supports: `*it` (read), `++it`, `it==it2`, `it!=it2`
- Example: `std::istream_iterator`

OUTPUT ITERATOR

- Write-only, single-pass (like writing to a file)
- Supports: `*it = val` (write), `++it`
- Example: `std::ostream_iterator`

FORWARD ITERATOR

- Read/write, can traverse multiple times, one direction
- Includes: Input + multi-pass guarantee
- Example: `std::forward_list` iterator

BIDIRECTIONAL ITERATOR

- Forward + can go backward (`--it`)
- Example: `std::list`, `std::map`, `std::set` iterators

RANDOM ACCESS ITERATOR

- Bidirectional + jump to any position in $O(1)$
- Supports: `it+n`, `it-n`, `it[n]`, `it1<it2`
- Example: `std::vector`, `std::deque` iterators
- Most powerful – works with all STL algorithms

CONTIGUOUS ITERATOR (C++17)

- Random Access + elements are contiguous in memory
- Example: `std::vector` iterator, `std::array` iterator

Complete Iterator Program

```
#include <iostream>
#include <vector>
#include <list>
#include <map>
#include <set>
#include <iterator> // std::advance, std::distance, std::next, std::prev
#include <algorithm>
#include <numeric>
#include <string>

int main() {
    std::cout << "┌───────────────────────────────────┐\n";
    std::cout << "│           Iterators – Complete Demo           │\n";
    std::cout << "└───────────────────────────────────┘\n\n";

    // — Basic Iterator Usage —————
    std::cout << "—— Basic iterator usage ——\n";
    std::vector<int> v = {10, 20, 30, 40, 50};

    // Traditional iterator loop
    std::cout << "Traditional: ";
    for (std::vector<int>::iterator it = v.begin(); it != v.end(); ++it)
        std::cout << *it << " ";
    std::cout << "\n";

    // 'auto' makes it cleaner
    std::cout << "With auto:   ";
    for (auto it = v.begin(); it != v.end(); ++it)
        std::cout << *it << " ";
    std::cout << "\n";

    // Range-based for (internally uses iterators)
    std::cout << "Range-for:   ";
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";

    // — Const iterators (read-only) —————
    std::cout << "\n—— const_iterator (read-only) ——\n";
    for (auto cit = v.cbegin(); cit != v.cend(); ++cit) {
        // *cit = 99; ← COMPILE ERROR: cit is const
    }
}
```

```

        std::cout << *cit << " ";
    }
    std::cout << "\n";

// — Reverse iterators —————
std::cout << "\n— Reverse iterators —\n";
std::cout << "Forward: ";
for (auto it = v.begin(); it != v.end(); ++it)
    std::cout << *it << " ";
std::cout << "\n";

std::cout << "Backward: ";
for (auto rit = v.rbegin(); rit != v.rend(); ++rit)
    std::cout << *rit << " ";
std::cout << "\n";

// — Random Access Iterator operations (vector/deque only) —————
std::cout << "\n— Random Access Iterator arithmetic —\n";
auto it = v.begin();
std::cout << "*begin()          = " << *it          << "\n";

it += 3;
std::cout << "after +=3:      = " << *it          << "\n";

--it;
std::cout << "after --:        = " << *it          << "\n";

std::cout << "it[0]            = " << it[0]          << "\n"; // same as *it
std::cout << "it[2]            = " << it[2]          << "\n"; // *(it+2)

// Distance between iterators
auto itA = v.begin();
auto itB = v.begin() + 4;
std::cout << "distance(begin,begin+4) = "
    << std::distance(itA, itB) << "\n";

// — std::advance and std::next / std::prev —————
std::cout << "\n— std::advance / next / prev —\n";
std::list<int> lst = {1, 2, 3, 4, 5, 6, 7};

auto lit = lst.begin();
std::advance(lit, 3); // move forward 3
std::cout << "after advance(3): " << *lit << "\n";

```

```

auto nextIt = std::next(lit, 2);          // 2 ahead, doesn't modify lit
std::cout << "next(+2):          " << *nextIt << "\n";

auto prevIt = std::prev(lit, 1);          // 1 behind
std::cout << "prev(-1):          " << *prevIt << "\n";

// — Bidirectional Iterator with list —————
std::cout << "\n— Bidirectional: list iterator forward + backward —\n";
std::cout << "Forward: ";
for (auto i = lst.begin(); i != lst.end(); ++i) std::cout << *i << " ";
std::cout << "\n";

std::cout << "Backward:";
for (auto i = lst.end(); i != lst.begin(); ) {
    --i;
    std::cout << *i << " ";
}
std::cout << "\n";

// — Iterator invalidation warning —————
std::cout << "\n— Iterator invalidation —\n";
std::vector<int> vv = {1, 2, 3, 4, 5};
auto itSaved = vv.begin() + 2; // points to element 3
std::cout << "Saved iterator points to: " << *itSaved << "\n";
// vv.push_back(6); ← MAY invalidate all iterators (reallocation)!
// SAFE: use vv.reserve() first, or re-obtain iterator after modification.

// — Stream iterators —————
std::cout << "\n— Stream iterators —\n";
std::vector<int> src = {5, 3, 8, 1, 9, 2};
std::cout << "Elements via ostream_iterator: ";
std::copy(src.begin(), src.end(),
           std::ostream_iterator<int>(std::cout, " "));
std::cout << "\n";

// — Insert iterators —————
std::cout << "\n— Insert iterators —\n";
std::vector<int> dest;
std::copy(src.begin(), src.end(),
           std::back_inserter(dest)); // back_insert_iterator
std::cout << "Copied via back_inserter: ";
for (int x : dest) std::cout << x << " ";

```

```

std::cout << "\n";

std::vector<int> dest2;
dest2.resize(src.size());
std::copy(src.begin(), src.end(), dest2.begin()); // regular iterator
std::sort(dest2.begin(), dest2.end());
std::cout << "Sorted copy:          ";
for (int x : dest2) std::cout << x << " ";
std::cout << "\n";

// — Iterator with map —————
std::cout << "\n— Map iterators (pairs) —\n";
std::map<std::string, int> m = {{ "Alice", 92 }, { "Bob", 78 }, { "Carol", 95 }};

// Each map iterator dereferences to std::pair<const K, V>
for (auto it2 = m.begin(); it2 != m.end(); ++it2) {
    std::cout << it2->first << " → " << it2->second << "\n";
}

// Reverse map iteration (highest key last → reversed = highest first)
std::cout << "Reversed: ";
for (auto rit = m.rbegin(); rit != m.rend(); ++rit)
    std::cout << rit->first << "=" << rit->second << " ";
std::cout << "\n";

return 0;
}

```

23.4 STL Algorithms

Theory

`<algorithm>` provides over 100 ready-made algorithms that work with any range defined by two iterators. Because they operate on iterators (not containers), they are fully **container-agnostic** — the same `std::sort` works on a `vector`, a `deque`, or a raw array.

THE ALGORITHM DESIGN PHILOSOPHY:

All algorithms take: (first_iterator, last_iterator [, extra args])

They never touch the container directly – only through iterators.

This means you can apply ANY algorithm to ANY range.

ALGORITHM CATEGORIES:

Non-modifying: find, count, search, for_each, equal, mismatch

Modifying: copy, move, fill, replace, transform, remove

Sorting: sort, stable_sort, partial_sort, nth_element

Binary search: binary_search, lower_bound, upper_bound

Set operations: set_union, set_intersection, set_difference

Min/Max: min_element, max_element, minmax_element

Numeric: accumulate, inner_product, partial_sum (in <numeric>)

Complete STL Algorithms Program

```
#include <iostream>
#include <vector>
#include <list>
#include <algorithm>
#include <numeric>
#include <functional>
#include <string>
#include <iterator>

void printVec(const std::vector<int>& v, const std::string& lbl) {
    std::cout << lbl << ": ";
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";
}

int main() {
    std::cout << "┌───────────────────────────────────────────┐\n";
    std::cout << "│      STL Algorithms – Complete Demo      │\n";
    std::cout << "└───────────────────────────────────────────┘\n\n";

    std::vector<int> v = {5, 2, 8, 1, 9, 3, 7, 4, 6, 1, 3};
    printVec(v, "original");

    // — for_each: apply a function to every element —————
    std::cout << "\n— for_each —\n";
    int runningSum = 0;
    std::for_each(v.begin(), v.end(), [&](int x) { runningSum += x; });
    std::cout << "Sum via for_each: " << runningSum << "\n";

    // for_each to print with formatting
    std::cout << "Squares:          ";
    std::for_each(v.begin(), v.end(), [](int x) {
        std::cout << x*x << " ";
    });
    std::cout << "\n";

    // — find: find first occurrence —————
    std::cout << "\n— find —\n";
    auto it = std::find(v.begin(), v.end(), 7);
    if (it != v.end())
```

```

    std::cout << "Found 7 at index "
                << std::distance(v.begin(), it) << "\n";

    auto notFound = std::find(v.begin(), v.end(), 99);
    std::cout << "Find 99: " << (notFound == v.end() ? "not found" : "found") << "\n";

    // find_if: find first element matching a predicate
    auto firstEven = std::find_if(v.begin(), v.end(),
                                   [](int x){ return x % 2 == 0; });
    std::cout << "First even: " << *firstEven << "\n";

    // find_if_not: first element NOT matching
    auto firstOdd = std::find_if_not(v.begin(), v.end(),
                                      [](int x){ return x % 2 == 0; });
    std::cout << "First odd: " << *firstOdd << "\n";

    // — count / count_if —————
    std::cout << "\n— count / count_if —\n";
    int cnt1 = std::count(v.begin(), v.end(), 3);
    std::cout << "Count of 3:      " << cnt1 << "\n";

    int cntEven = std::count_if(v.begin(), v.end(),
                                [](int x){ return x % 2 == 0; });
    std::cout << "Count of evens: " << cntEven << "\n";

    int cntGt5 = std::count_if(v.begin(), v.end(),
                                [](int x){ return x > 5; });
    std::cout << "Count > 5:      " << cntGt5 << "\n";

    // — sort —————
    std::cout << "\n— sort —\n";
    std::vector<int> sv = v;    // make a copy to sort
    printVec(sv, "before sort ");
    std::sort(sv.begin(), sv.end());
    printVec(sv, "ascending  ");
    std::sort(sv.begin(), sv.end(), std::greater<int>());
    printVec(sv, "descending ");

    // stable_sort: preserves relative order of equal elements
    std::vector<std::pair<int, std::string>> students =
        {{3, "Alice"}, {1, "Bob"}, {2, "Carol"}, {1, "Diana"}, {2, "Eve"}};
    std::stable_sort(students.begin(), students.end(),
                    [](const auto& a, const auto& b){ return a.first < b.first; });

```

```

std::cout << "stable_sort by grade:\n";
for (const auto& [g, name] : students)
    std::cout << "  grade=" << g << " " << name << "\n";

// — search: find a subsequence —————
std::cout << "\n— search —\n";
std::vector<int> haystack = {1,2,3,4,5,6,7,8};
std::vector<int> needle   = {4,5,6};
auto pos = std::search(haystack.begin(), haystack.end(),
                      needle.begin(), needle.end());
if (pos != haystack.end())
    std::cout << "Found subsequence {4,5,6} at index "
               << std::distance(haystack.begin(), pos) << "\n";

// — copy —————
std::cout << "\n— copy / copy_if / copy_n —\n";
std::vector<int> src = {1,2,3,4,5,6,7,8,9,10};
std::vector<int> dst;

// copy_if: copy only even numbers
std::copy_if(src.begin(), src.end(), std::back_inserter(dst),
             [](int x){ return x % 2 == 0; });
printVec(dst, "copy_if (evens)");

// copy_n: copy first 5 elements
std::vector<int> dst2(5);
std::copy_n(src.begin(), 5, dst2.begin());
printVec(dst2, "copy_n (first 5)");

// — transform: apply function to each element —————
std::cout << "\n— transform —\n";
std::vector<int> orig = {1,2,3,4,5};
std::vector<int> squared(orig.size());
std::transform(orig.begin(), orig.end(), squared.begin(),
               [](int x){ return x * x; });
printVec(orig, "original");
printVec(squared, "squared ");

// transform with two input ranges: element-wise addition
std::vector<int> a = {1,2,3,4,5};
std::vector<int> b = {10,20,30,40,50};
std::vector<int> result(5);
std::transform(a.begin(), a.end(), b.begin(), result.begin(),

```

```

        [](int x, int y){ return x + y; });
printVec(result, "a+b      ");

// — remove / erase (erase-remove idiom) —————
std::cout << "\n— remove / erase-remove idiom —\n";
std::vector<int> rv = {1,2,3,2,4,2,5,2};
printVec(rv, "before remove(2)");

// std::remove does NOT erase – it moves non-matching elements forward
// and returns iterator to the new "logical end"
auto newEnd = std::remove(rv.begin(), rv.end(), 2);
std::cout << "After std::remove – logical end at index "
          << std::distance(rv.begin(), newEnd) << "\n";
printVec(rv, "after remove (no erase yet)");

// erase removes the "garbage" tail
rv.erase(newEnd, rv.end());
printVec(rv, "after erase      ");

// remove_if: remove elements matching a predicate
std::vector<int> rv2 = {1,2,3,4,5,6,7,8,9,10};
rv2.erase(std::remove_if(rv2.begin(), rv2.end(),
    [](int x){ return x % 3 == 0; }),
    rv2.end());
printVec(rv2, "removed multiples of 3");

// — fill / generate —————
std::cout << "\n— fill / generate —\n";
std::vector<int> fv(8);
std::fill(fv.begin(), fv.end(), 7);
printVec(fv, "fill(7)");

int counter = 0;
std::generate(fv.begin(), fv.end(), [&]() { return counter++ * 10; });
printVec(fv, "generate");

// — unique: remove CONSECUTIVE duplicates —————
std::cout << "\n— unique —\n";
std::vector<int> uv = {1,1,2,3,3,3,4,4,5};
printVec(uv, "before unique");
uv.erase(std::unique(uv.begin(), uv.end()), uv.end());
printVec(uv, "after unique ");

```

```
// — min_element / max_element —————
std::cout << "\n— min/max/minmax —\n";
std::vector<int> mv = {5,3,8,1,9,2};
auto minIt = std::min_element(mv.begin(), mv.end());
auto maxIt = std::max_element(mv.begin(), mv.end());
auto [mn, mx] = std::minmax_element(mv.begin(), mv.end());
std::cout << "min=" << *minIt << " max=" << *maxIt
           << " minmax=(" << *mn << ", " << *mx << ")\n";
```

```
// — Numeric algorithms (from <numeric>) —————
std::cout << "\n— Numeric algorithms —\n";
std::vector<int> nv = {1,2,3,4,5,6,7,8,9,10};
int total = std::accumulate(nv.begin(), nv.end(), 0);
int product = std::accumulate(nv.begin(), nv.end(), 1,
                             std::multiplies<int>());
std::cout << "accumulate (sum): " << total << "\n";
std::cout << "accumulate (product): " << product << "\n";
```

```
std::vector<int> partSums(nv.size());
std::partial_sum(nv.begin(), nv.end(), partSums.begin());
printVec(partSums, "partial_sum (prefix sums)");
```

```
std::vector<int> diffs(nv.size());
std::adjacent_difference(nv.begin(), nv.end(), diffs.begin());
printVec(diffs, "adjacent_difference ");
```

```
// — all_of / any_of / none_of —————
std::cout << "\n— all_of / any_of / none_of —\n";
std::vector<int> pv = {2,4,6,8,10};
std::cout << "all even: " << (std::all_of (pv.begin(),pv.end(),[](int x){return x%2==0;})) << "\n";
std::cout << "any > 5: " << (std::any_of (pv.begin(),pv.end(),[](int x){return x>5;})) << "\n";
std::cout << "none neg: " << (std::none_of(pv.begin(),pv.end(),[](int x){return x<0;})) << "\n";
```

```
return 0;
```

```
}
```

23.5 Adaptors

Theory

A **container adaptor** is a class that wraps an existing STL container and restricts its interface to provide a specific behaviour. The adaptor does not implement storage itself — it **delegates** to an underlying container.

ADAPTOR CONCEPT:

```
template <typename T, typename Container = std::deque<T>>
class stack {
    Container c;    // ← the actual storage (default: deque)
public:
    void push(const T& v) { c.push_back(v); }
    void pop()           { c.pop_back(); }
    T& top()             { return c.back(); }
    bool empty()         { return c.empty(); }
    size_t size()        { return c.size(); }
    // NO begin(), end(), random access – deliberately restricted!
};
```

AVAILABLE ADAPTORS:

```
std::stack<T, Container>           → LIFO (default: deque)
std::queue<T, Container>          → FIFO (default: deque)
std::priority_queue<T, Container> → highest priority first (default: vector)
```

Adaptors and Priority Queue — Complete Program

```
#include <iostream>
#include <stack>
#include <queue>
#include <vector>
#include <list>
#include <string>
#include <functional> // std::greater

int main() {
    std::cout << "┌───────────────────────────────────┐\n";
    std::cout << "│      Container Adaptors – Complete Demo      │\n";
    std::cout << "└───────────────────────────────────┘\n\n";

    // — stack with different underlying containers —————
    std::cout << "— stack<int> (default: deque) —\n";
    std::stack<int> s1; // default: deque
    std::stack<int, std::vector<int>> s2; // backed by vector
    std::stack<int, std::list<int>> s3; // backed by list

    for (int i : {10,20,30,40,50}) s1.push(i);
    std::cout << "s1 (deque-backed): top=" << s1.top()
               << " size=" << s1.size() << "\n";

    for (int i : {10,20,30,40,50}) s2.push(i);
    std::cout << "s2 (vector-backed): top=" << s2.top()
               << " size=" << s2.size() << "\n";

    // — queue with list as underlying container —————
    std::cout << "\n— queue<int, list<int>> —\n";
    std::queue<int, std::list<int>> q;
    for (int i : {1,2,3,4,5}) q.push(i);
    std::cout << "front=" << q.front() << " back=" << q.back()
               << " size=" << q.size() << "\n";
    while (!q.empty()) {
        std::cout << q.front() << " "; q.pop();
    }
    std::cout << "\n";

    // — priority_queue (max-heap by default) —————
    std::cout << "\n— priority_queue (max-heap) —\n";
```

```

std::priority_queue<int> maxPQ;
for (int x : {3,1,4,1,5,9,2,6,5,3}) maxPQ.push(x);

std::cout << "Draining max-heap (largest first): ";
while (!maxPQ.empty()) {
    std::cout << maxPQ.top() << " ";
    maxPQ.pop();
}
std::cout << "\n";

// — priority_queue (min-heap using greater<>) —————
std::cout << "\n— priority_queue (min-heap) —\n";
std::priority_queue<int, std::vector<int>, std::greater<int>> minPQ;
for (int x : {3,1,4,1,5,9,2,6,5,3}) minPQ.push(x);

std::cout << "Draining min-heap (smallest first): ";
while (!minPQ.empty()) {
    std::cout << minPQ.top() << " ";
    minPQ.pop();
}
std::cout << "\n";

// — priority_queue with custom struct —————
std::cout << "\n— priority_queue with custom comparator —\n";

struct Task {
    std::string name;
    int         priority;    // higher = served first
    int         arrival;     // for tie-breaking
};

// Custom comparator: higher priority first; ties broken by arrival
auto cmp = [](const Task& a, const Task& b) {
    if (a.priority != b.priority) return a.priority < b.priority;
    return a.arrival > b.arrival;    // lower arrival = served first
};

std::priority_queue<Task, std::vector<Task>, decltype(cmp)> taskQ(cmp);

taskQ.push({"Fix login bug",      5, 1});
taskQ.push({"Update docs",       1, 2});
taskQ.push({"Deploy hotfix",     5, 3});
taskQ.push({"Code review",       3, 4});

```

```

taskQ.push({"Performance tuning", 4, 5});
taskQ.push({"UI redesign", 2, 6});

std::cout << "Processing tasks by priority:\n";
int order = 1;
while (!taskQ.empty()) {
    const Task& t = taskQ.top();
    std::cout << "  " << order++ << ". [P=" << t.priority << "] "
                << t.name << "\n";
    taskQ.pop();
}

// — Custom adaptor: LimitedStack —————
// An adaptor that limits the stack to a maximum size.
std::cout << "\n— Custom Adaptor: LimitedStack<T, MAX> —\n";

template_not_needed; // label trick to allow 'goto' past lambdas

// Simulate a limited stack as a local class
class LimitedStack {
    std::stack<int> inner;
    int maxSize;
public:
    LimitedStack(int max) : maxSize(max) {}
    bool push(int val) {
        if ((int)inner.size() >= maxSize) {
            std::cout << "  LimitedStack full! Cannot push " << val << "\n";
            return false;
        }
        inner.push(val);
        std::cout << "  pushed " << val
                  << "  [" << inner.size() << "/" << maxSize << "]\n";
        return true;
    }
    void pop() {
        if (inner.empty()) { std::cout << "  Empty!\n"; return; }
        std::cout << "  popped " << inner.top() << "\n";
        inner.pop();
    }
    int top() const { return inner.top(); }
    bool empty() const { return inner.empty(); }
    int size() const { return inner.size(); }
};

```

```

LimitedStack ls(3);
ls.push(10);
ls.push(20);
ls.push(30);
ls.push(40);    // should fail
ls.pop();
ls.push(40);    // now succeeds
ls.pop(); ls.pop(); ls.pop();
ls.pop();    // empty

// — Real-world: Dijkstra's algorithm uses priority_queue —————
std::cout << "\n— Real-world: Shortest path (Dijkstra sketch) —\n";
// Adjacency list: {neighbour, weight}
const int N = 5;
std::vector<std::pair<int,int>> graph[N];
graph[0].push_back({1, 4}); graph[0].push_back({2, 1});
graph[1].push_back({3, 1});
graph[2].push_back({1, 2}); graph[2].push_back({3, 5});
graph[3].push_back({4, 3});

std::vector<int> dist(N, INT_MAX);
dist[0] = 0;

// min-heap: {distance, node}
std::priority_queue<std::pair<int,int>,
                    std::vector<std::pair<int,int>>,
                    std::greater<std::pair<int,int>>> pq;
pq.push({0, 0});

while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;    // stale entry
    for (auto [v, w] : graph[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v});
        }
    }
}

std::cout << "Shortest distances from node 0:\n";
for (int i = 0; i < N; i++) {

```

```

        std::cout << " → node " << i << ": "
            << (dist[i] == INT_MAX ? -1 : dist[i]) << "\n";
    }

    return 0;
}

```

23.6 Allocators

Theory

Every STL container manages its own memory. By default it uses `std::allocator<T>`, which calls `::operator new` and `::operator delete` — the standard global heap. An **allocator** is the policy object that a container uses to obtain and release memory.

WHY CUSTOM ALLOCATORS?

DEFAULT: Container calls `::new/::delete` for every element.

Problem areas:

- High-frequency small allocations → heap fragmentation
- Embedded systems with no heap (use static pool instead)
- Need exact control over where memory comes from (NUMA, GPU, shared mem)
- Performance-critical code (arena allocator = zero fragmentation, $O(1)$)
- Memory debugging (track every allocation)

CUSTOM ALLOCATOR: you provide the memory sourcing strategy.

The container calls your allocator instead of the global heap.

ALLOCATOR CONCEPT (minimum interface):

```
template <typename T>
class MyAllocator {
public:
    using value_type = T;

    MyAllocator() noexcept;

    template <typename U>
    MyAllocator(const MyAllocator<U>&) noexcept;

    T*   allocate(std::size_t n);      // allocate n objects of type T
    void deallocate(T* ptr, std::size_t n); // deallocate
};
```

Complete Allocator Program

```
#include <iostream>
#include <vector>
#include <list>
#include <memory>      // std::allocator
#include <limits>
#include <string>

// =====
// 1. LOGGING ALLOCATOR: wraps std::allocator and logs every operation
// =====

template <typename T>
class LoggingAllocator {
public:
    using value_type = T;

    LoggingAllocator() noexcept = default;

    // Required: rebind constructor (allows vector to rebind to Node type)
    template <typename U>
    LoggingAllocator(const LoggingAllocator<U>&) noexcept {}

    T* allocate(std::size_t n) {
        std::size_t bytes = n * sizeof(T);
        T* ptr = static_cast<T*>(::operator new(bytes));
        std::cout << " [LogAlloc] allocate " << n
                    << " x " << sizeof(T) << " bytes"
                    << " = " << bytes << " bytes"
                    << " @ " << static_cast<void*>(ptr) << "\n";
        return ptr;
    }

    void deallocate(T* ptr, std::size_t n) noexcept {
        std::cout << " [LogAlloc] deallocate " << n
                    << " x " << sizeof(T) << " bytes"
                    << " @ " << static_cast<void*>(ptr) << "\n";
        ::operator delete(ptr);
    }

    // Equality: two LoggingAllocators are always interchangeable
```

```

    bool operator==(const LoggingAllocator&) const noexcept { return true; }
    bool operator!=(const LoggingAllocator&) const noexcept { return false; }
};

// =====
// 2. POOL ALLOCATOR: allocates from a fixed-size static pool
//     No heap calls after construction – ideal for embedded systems.
// =====

template <typename T, std::size_t POOL_SIZE = 64>
class PoolAllocator {
public:
    using value_type = T;

    // Static pool: one shared pool per (T, POOL_SIZE) combination
    static unsigned char pool[POOL_SIZE * sizeof(T)];
    static bool          used[POOL_SIZE];
    static bool          poolInitialised;

    PoolAllocator() noexcept {
        if (!poolInitialised) {
            for (bool& b : used) b = false;
            poolInitialised = true;
        }
    }

    template <typename U>
    PoolAllocator(const PoolAllocator<U, POOL_SIZE>&) noexcept {}

    T* allocate(std::size_t n) {
        // Find n consecutive free slots
        for (std::size_t i = 0; i + n <= POOL_SIZE; i++) {
            bool ok = true;
            for (std::size_t j = i; j < i + n; j++)
                if (used[j]) { ok = false; break; }

            if (ok) {
                for (std::size_t j = i; j < i + n; j++) used[j] = true;
                T* ptr = reinterpret_cast<T*>(pool + i * sizeof(T));
                std::cout << " [PoolAlloc] allocate " << n
                          << " object(s) at slot " << i
                          << " (" << n*sizeof(T) << " bytes)\n";
                return ptr;
            }
        }
    }
};

```

```

    }
}
throw std::bad_alloc();
}

void deallocate(T* ptr, std::size_t n) noexcept {
    T* base = reinterpret_cast<T*>(pool);
    auto slot = static_cast<std::size_t>(ptr - base);
    std::cout << " [PoolAlloc] deallocate " << n
                << " object(s) from slot " << slot << "\n";
    for (std::size_t j = slot; j < slot + n; j++) used[j] = false;
}

bool operator==(const PoolAllocator&) const noexcept { return true; }
bool operator!=(const PoolAllocator&) const noexcept { return false; }
};

// Static member definitions
template <typename T, std::size_t S>
unsigned char PoolAllocator<T,S>::pool[S * sizeof(T)];

template <typename T, std::size_t S>
bool PoolAllocator<T,S>::used[S];

template <typename T, std::size_t S>
bool PoolAllocator<T,S>::poolInitialised = false;

// =====
// MAIN
// =====

int main() {
    std::cout << "===== \n";
    std::cout << "|| Allocators - Complete Demo || \n";
    std::cout << "===== \n\n";

    // — 1. Default allocator (for reference) —————
    std::cout << "— Default allocator (std::allocator) — \n";
    {
        std::vector<int> v = {1, 2, 3};
        v.push_back(4);
        std::cout << "Default vector: size=" << v.size() << " cap=" << v.capacity() << "\n";
    }
}

```

```

std::cout << "\n";

// — 2. Logging allocator with vector —————
std::cout << "—— LoggingAllocator with vector<int> ——\n";
{
    // vector<int, LoggingAllocator<int>>
    std::vector<int, LoggingAllocator<int>> logVec;
    std::cout << "Pushing 5 elements:\n";
    for (int i = 1; i <= 5; i++) logVec.push_back(i * 10);
    std::cout << "Content: ";
    for (int x : logVec) std::cout << x << " ";
    std::cout << "\n";
    // Destructor will trigger a deallocate
    std::cout << "Leaving scope → vector destructs:\n";
}
std::cout << "\n";

// — 3. Logging allocator with reserve —————
std::cout << "—— LoggingAllocator + reserve (single allocation) ——\n";
{
    std::vector<int, LoggingAllocator<int>> logVec2;
    logVec2.reserve(10); // ONE allocation upfront
    std::cout << "After reserve(10), now pushing:\n";
    for (int i = 1; i <= 8; i++) logVec2.push_back(i);
    std::cout << "No reallocation because reserved enough!\n";
    std::cout << "Leaving scope:\n";
}
std::cout << "\n";

// — 4. Pool allocator with vector —————
std::cout << "—— PoolAllocator<int, 16> with vector<int> ——\n";
{
    std::vector<int, PoolAllocator<int, 16>> poolVec;
    poolVec.reserve(5); // allocate from pool upfront
    for (int i : {100, 200, 300, 400, 500})
        poolVec.push_back(i);

    std::cout << "Pool vector content: ";
    for (int x : poolVec) std::cout << x << " ";
    std::cout << "\n";
    std::cout << "Leaving scope → deallocation from pool:\n";
}
std::cout << "\n";

```

// — 5. std::allocator direct usage —————

```
std::cout << "—— std::allocator<int> used directly ——\n";
{
    std::allocator<int> alloc;

    // Allocate raw memory for 5 ints
    int* raw = alloc.allocate(5);
    std::cout << "Allocated 5 ints at " << raw << "\n";

    // Construct objects in the allocated memory
    for (int i = 0; i < 5; i++) {
        new (raw + i) int(i * 11);    // placement new
    }

    std::cout << "Constructed values: ";
    for (int i = 0; i < 5; i++) std::cout << raw[i] << " ";
    std::cout << "\n";

    // Destroy objects
    for (int i = 0; i < 5; i++) raw[i].~int();    // trivial for int

    // Deallocate raw memory
    alloc.deallocate(raw, 5);
    std::cout << "Deallocated.\n\n";
}
```

// — 6. Why reserve() matters (with logging) —————

```
std::cout << "—— Push_back without reserve (multiple reallocations) ——\n";
{
    std::vector<int, LoggingAllocator<int>> v;
    std::cout << "Pushing 8 elements without reserve:\n";
    for (int i = 1; i <= 8; i++) v.push_back(i);
    std::cout << "Leaving scope:\n";
}
```

```
std::cout << "\n—— Push_back WITH reserve (single allocation) ——\n";
```

```
{
    std::vector<int, LoggingAllocator<int>> v;
    v.reserve(8);
    std::cout << "Pushing 8 elements after reserve(8):\n";
    for (int i = 1; i <= 8; i++) v.push_back(i);
    std::cout << "Leaving scope:\n";
}
```

```
}

return 0;
}
```

Chapter 23 — Common Mistakes

CONTAINERS:

- ✗ Using operator[] on map when checking existence
scores["Zara"]; // INSERTS "Zara" with default value 0!
Fix: use scores.find("Zara") or scores.count("Zara")
- ✗ Invalidating iterators and continuing to use them
for (auto it = v.begin(); it != v.end(); ++it)
 v.push_back(10); // push_back may reallocate → it is INVALID!
Fix: collect indices or use indices instead of iterators during modification.
- ✗ Forgetting the erase-remove idiom
std::remove(v.begin(), v.end(), 3); // does NOT shrink the vector!
Fix: v.erase(std::remove(v.begin(), v.end(), 3), v.end());
- ✗ Using std::sort on a list (it doesn't have random-access iterators)
std::sort(lst.begin(), lst.end()); // COMPILE ERROR
Fix: lst.sort()

ITERATORS:

- ✗ Dereferencing end() iterator
*v.end() // UB: end() is one-past-the-last, not a valid element

ALGORITHMS:

- ✗ Passing wrong order of iterators
std::sort(v.end(), v.begin()); // UB: reversed range

ALLOCATORS:

- ✗ Forgetting that pool/arena allocators may not support arbitrary sizes
Pool of 16 ints cannot hold a 17th element – check before allocating.

Best Practices

- ✓ Default to `std::vector` – it's the fastest for most use cases
- ✓ Use `std::unordered_map` over `std::map` when you don't need sorted order
- ✓ Always `reserve()` vector capacity if you know the final size
- ✓ Use the erase-remove idiom to delete elements from vector
- ✓ Prefer range-based for loops over raw iterator loops (cleaner)
- ✓ Use `cbegin()/cend()` for read-only iteration (communicates intent)
- ✓ Prefer `std::sort` (introsort – $O(n \log n)$ worst-case) over manual sorting
- ✓ Use structured bindings (C++17) with map: `for (auto& [k,v] : m)`
- ✓ Profile before using custom allocators – the default is often good enough

Quick Reference

STL CONTAINERS QUICK REFERENCE			
Container	Best use case		
vector	Default. Fast access + push_back. Most common.		
deque	Fast push/pop at BOTH ends + random access		
list	Frequent middle insert/delete; no random access		
map	Key-value, sorted keys, O(log n)		
unord_map	Key-value, unsorted, O(1) avg lookup		
set	Unique sorted elements		
unord_set	Unique elements, O(1) avg lookup		
Adaptor	Wraps	Purpose	
stack	deque (default)	LIFO	
queue	deque (default)	FIFO	
priority_q	vector (default)	Highest priority first	
COMMON INTERFACE: begin/end size empty clear insert erase			
ERASE-REMOVE: v.erase(std::remove(v.begin(),v.end(),val),v.end())			
SORT: std::sort(v.begin(), v.end())			
FIND: std::find(v.begin(), v.end(), val) != v.end()			

Additional Resources

Books

- **"The C++ Standard Library" by Nicolai Josuttis** — The definitive STL reference. Every container, iterator, algorithm, and allocator covered exhaustively.
- **"Effective STL" by Scott Meyers** — 50 specific rules for using the STL correctly and efficiently. Essential reading after learning the basics.
- **"C++ Primer" by Lippman et al.** — Chapters 9–11 give a beginner-accessible introduction to all the containers in this chapter.

Online

- cppreference.com/w/cpp/container — Complete reference for every STL container with complexity guarantees.
- cppreference.com/w/cpp/algorithm — Every algorithm listed with parameters, complexity, and examples.
- learncpp.com — Chapters 16–18 cover containers, iterators, and algorithms with guided examples.

 **Pro Tip:** The single most impactful STL habit is choosing the right container upfront. Use `std::vector` by default. Switch to `std::list` only when profiling shows that middle insertions dominate. Switch to `std::unordered_map` when lookup count dominates and ordering is irrelevant. The STL's power is not just in having these containers — it's in how they interoperate through iterators and algorithms.

End of Chapter 23: STL Containers in C++

Theory-first, complete runnable examples, beginner to expert

Guide Version 1.0 — using `iostream` throughout